



Linux Device Driver Development Cont.

ECE 391 – Computer Systems Engineering

Dong Kai Wang

Department of Electrical and Computer Engineering

University of Illinois at Urbana-Champaign

- **Driver Examples on Course Website**
- **Implementation of scull Driver**
 - Main data structures.
 - Implementation of scull functions: `open()` and `release()`.
 - Dynamic memory allocation.
 - Implementation of scull functions: `read()` and `write()`.
 - Implementation of scull functions: `ioctl()`.
 - Concurrency using semaphores.
- *Reference: Linux Device Drivers, 3rd Edition, O'Reilly Media.*

- Write a basic ***kernel module***:
 - Implement module `init` and `exit` functions.
 - Implement file operations functions (`read/write`).
- Register a ***major number*** for the device driver.
 - For character devices, we call `register_chrdev`.
 - Request specific major number or let Linux assign an unused number for us.
- Create a ***special file*** (e.g., using `mknod`) to allow user programs to interact with driver through system calls (`open/read/write/...` to the ***file***).
 - Multiple devices with different minor numbers can share the same driver (major number).
 - See the `readsimple` or `readwrite` driver demonstration.

- The `file_operations` struct contains many members, here are some:

`struct module *owner`

- Not an operation, but a pointer to the driver module that owns this struct.
- Prevent it from being unloaded when an operation is taking place.
- Set it using the macro `THIS_MODULE` defined in `<linux/module.h>`.

`ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);`

- Return number of **bytes successfully read** to user (`char __user *`) and update position.
- Read system call fails with `-EINVAL` if not implemented (passed `NULL` function pointer).

`ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);`

- Return number of **bytes successfully written** to device and update position.
- Write system call fails with `-EINVAL` if not implemented.

```
int (*open) (struct inode *, struct file *);
```

- Opens a file, parameters (inode) provided by kernel, not required to implement.

```
int (*release) (struct inode *, struct file *);
```

- Releases (last process closes) the file, also optional.

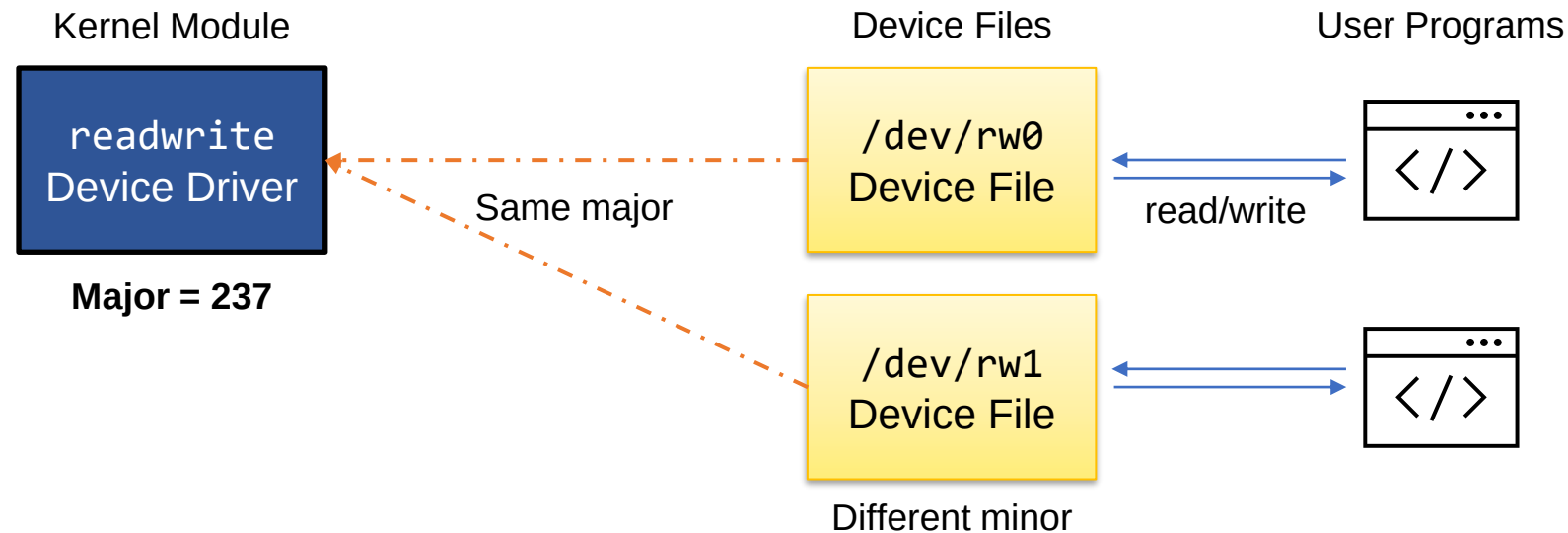
```
int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
```

- Issue device-specific commands (up to your implementation).
- E.g., can serve as alternative for module parameters.

```
loff_t (*llseek) (struct file *, loff_t, int);
```

- Change current read/write position within a file. The parameter long offset `loff_t` is at least 64 bits.

- *Recall:* multiple devices can have the same **major number** and share the same device driver.
- What happens when we create two special files with same major number.



- *What if we want separate data for different devices of the same type?*

- **Simple Character Utility for Loading Localities.**

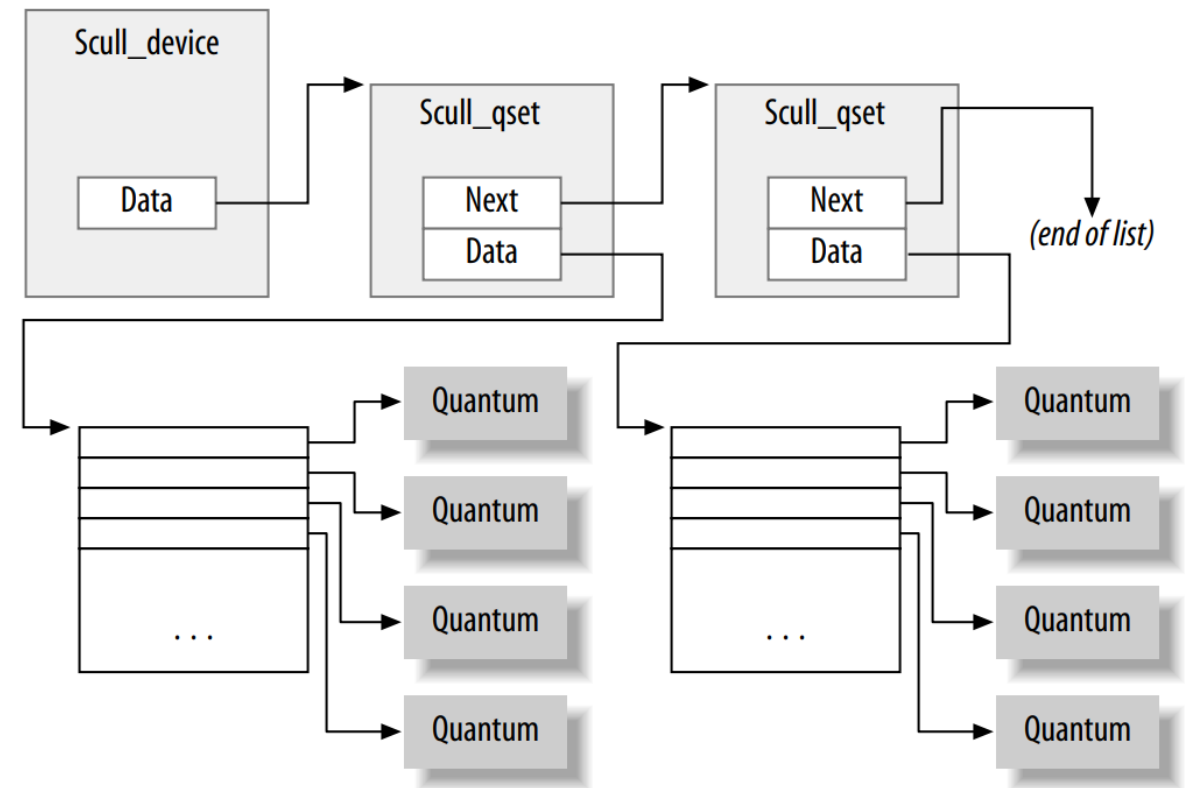
- A simple character device to store some data internally.
- *Memory-only* (does not act on actual hardware).
 - For portability reasons across platform.
 - We can assume it *represents* underlying hardware.

- Data structures: a linked list of pointer arrays.
 - The scull device contains multiple quantum sets.
 - Each quantum set is an array of pointers to data.

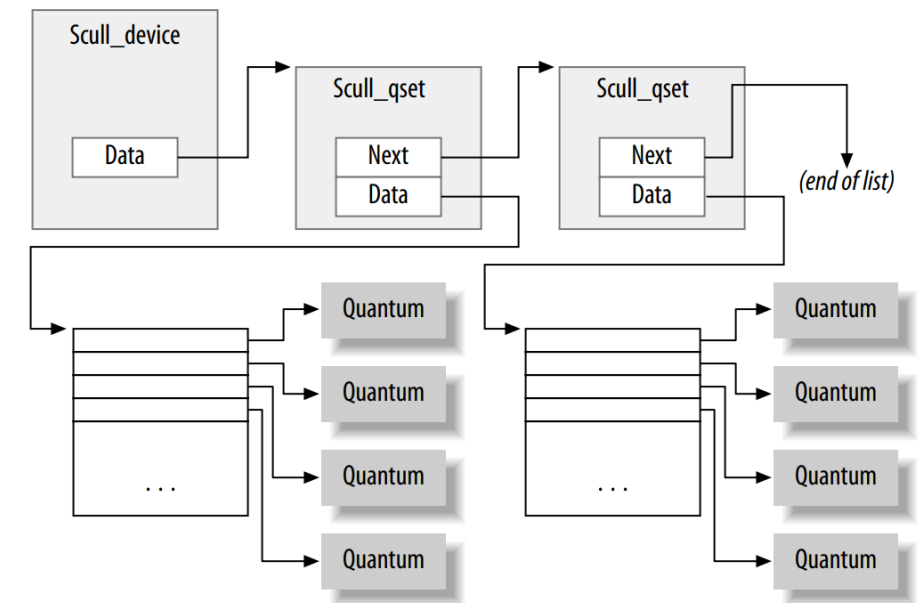
- **Modified scull driver on course site:**

- Should work on modern Linux kernels.

sculldriver.tar.xz (Try on your own VM)



- **Simple Character Utility for Loading Localities.**
- Dynamically allocates a linked list of “Quantum sets” in kernel memory.
 - Exists in memory only.
 - Designed to be portable across OS, so we can pretend they *represent* some underlying hardware.
 - Each Quantum set contains an array of Quantums, storing some data.
 - Each Quantum set contains 1000 pointers.
 - Each Quantum stores 4000 bytes.
- Showcases various features of drivers.
 - Memory allocation.
 - Race conditions.
 - Privileges and security.
 - And more ...



Recall that every process has a file struct * array (*fd*) in kernel that stores information about its **open** files, this is specific to the process. (Note: not the same as *libc* FILE* pointer)

```
struct file_operations *f_op;
```

- File operations associated with file.
- Set initially by kernel on open, can be substituted with other functions in your driver.

```
loff_t f_pos;
```

- Current read/write position within a file.
- Should be directly updated by `llseek` and indirectly updated by read/write file operations.

```
struct dentry *f_dentry;
```

- Stores directory entry structure associated with file. Often not used except for accessing inode (`filp->f_dentry->d_inode`), containing device file's major/minor numbers (`i_major(struct inode* inode)`).

```
void *private_data;
```

- Set to NULL by kernel on open (not always), you can have this pointer point to any data in your driver.

- In general, the `open()` function should:
 - **Check** device status and check for hardware errors, if applicable.
 - **Initialize** device if being opened for first time.
 - Keep in mind, multiple processes may have already opened the same device.
 - **Update the file operations pointer** `f_op` if necessary.
 - **Allocate** and fill any data structure to be put into `filp->private_data`.

```
int scull_open(struct inode *inode, struct file *filp)
{
    Scull_Dev *dev; /* device information */
    int num = NUM(inode->i_rdev);
    int type = TYPE(inode->i_rdev);    // inode->i_rdev is a dev_t type, usually a single int with upper bits denoting
                                     // major number, lower bits denoting minor number.
}
```

- In the `scull` driver, for convenience, the authors use the upper 4-bits to distinguish between different types of `scull` drivers, and the lower 4-bits to denote device number.

```
#define TYPE(dev) (MINOR(dev) >> 4) /* high nibble */
#define NUM(dev) (MINOR(dev) & 0xf) /* low nibble */
```

- We may want different file operation functions to handle different types of `scull`.

```
if (type > SCULL_MAX_TYPE) return -ENODEV;
filp->f_op = scull_fop_array[type];
return filp->f_op->open(inode, filp); /* dispatch to specific open */
```

Set file operations based on type (minor number). Then call open function based on type.

- Next, we set the private data:

- Each minor number has its own `scull_dev` struct.
- Thus, quantum data is not shared between different device files.

```
/* type 0, check the device number (unless private_data valid) */
dev = (Scull_Dev *)filp->private_data;
if (!dev) {
    if (num >= scull_nr_devs) return -ENODEV;
    dev = &scull_devices[num];
    filp->private_data = dev; /* for other methods */
}
```

Array of `Scull_dev`, each containing their own data.

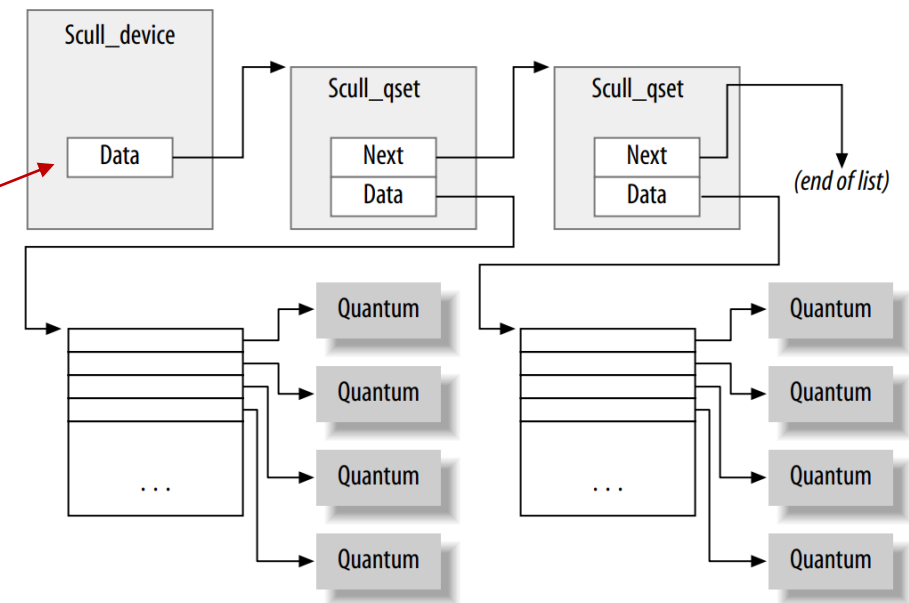
```
struct file_operations *scull_fop_array[]={
    &scull_fops, /* type 0 */
    &scull_priv_fops, /* type 1 */
    &scull_pipe_fops, /* type 2 */
    &scull_sngl_fops, /* type 3 */
    &scull_user_fops, /* type 4 */
    &scull_wusr_fops /* type 5 */
};
#define SCULL_MAX_TYPE 5
```

Array of file operation structs, each struct containing function pointers to its own `read()`, `write()`, `open()`, etc.

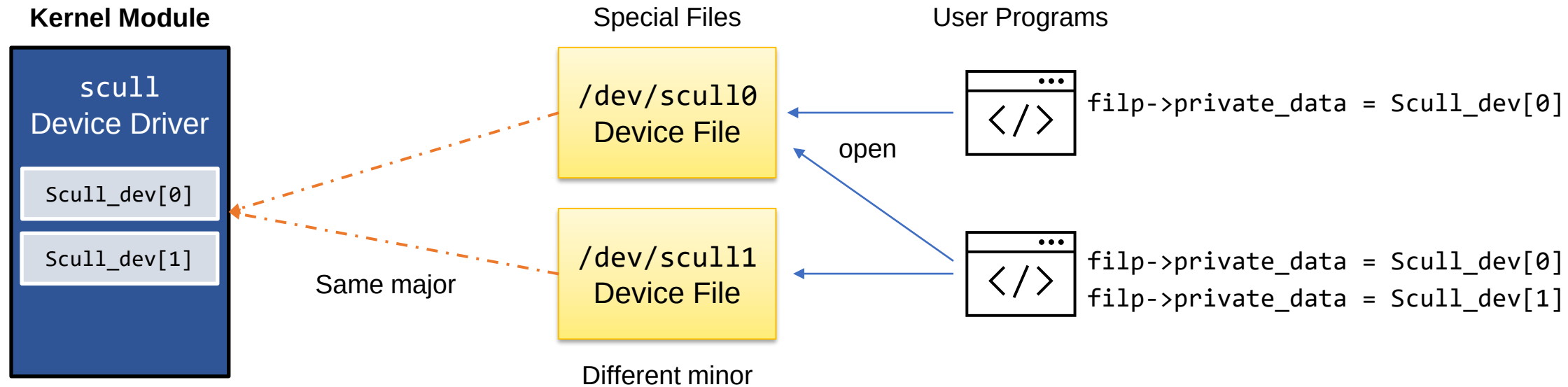
- Each device (minor number) has its own Scull_dev struct containing its data.
- Each process opening a device file will have its private_data pointer point to the Scull_dev struct based on the device file's **minor number**.

```
/* type 0, check the device number (unless private_data valid) */
dev = (Scull_Dev *)filp->private_data;
if (!dev) {
    if (num >= scull_nr_devs) return -ENODEV;
    dev = &scull_devices[num];
    filp->private_data = dev; /* for other methods */
}
```

```
struct scull_dev {
    struct scull_qset *data; /* Pointer to first quantum set */
    int quantum;             /* the current quantum size */
    int qset;                /* the current array size */
    unsigned long size;      /* amount of data stored here */
    unsigned int access_key; /* used by sculluid and scullpriv */
    struct semaphore sem;    /* mutual exclusion semaphore */
    struct cdev cdev;        /* Char device structure */
};
```

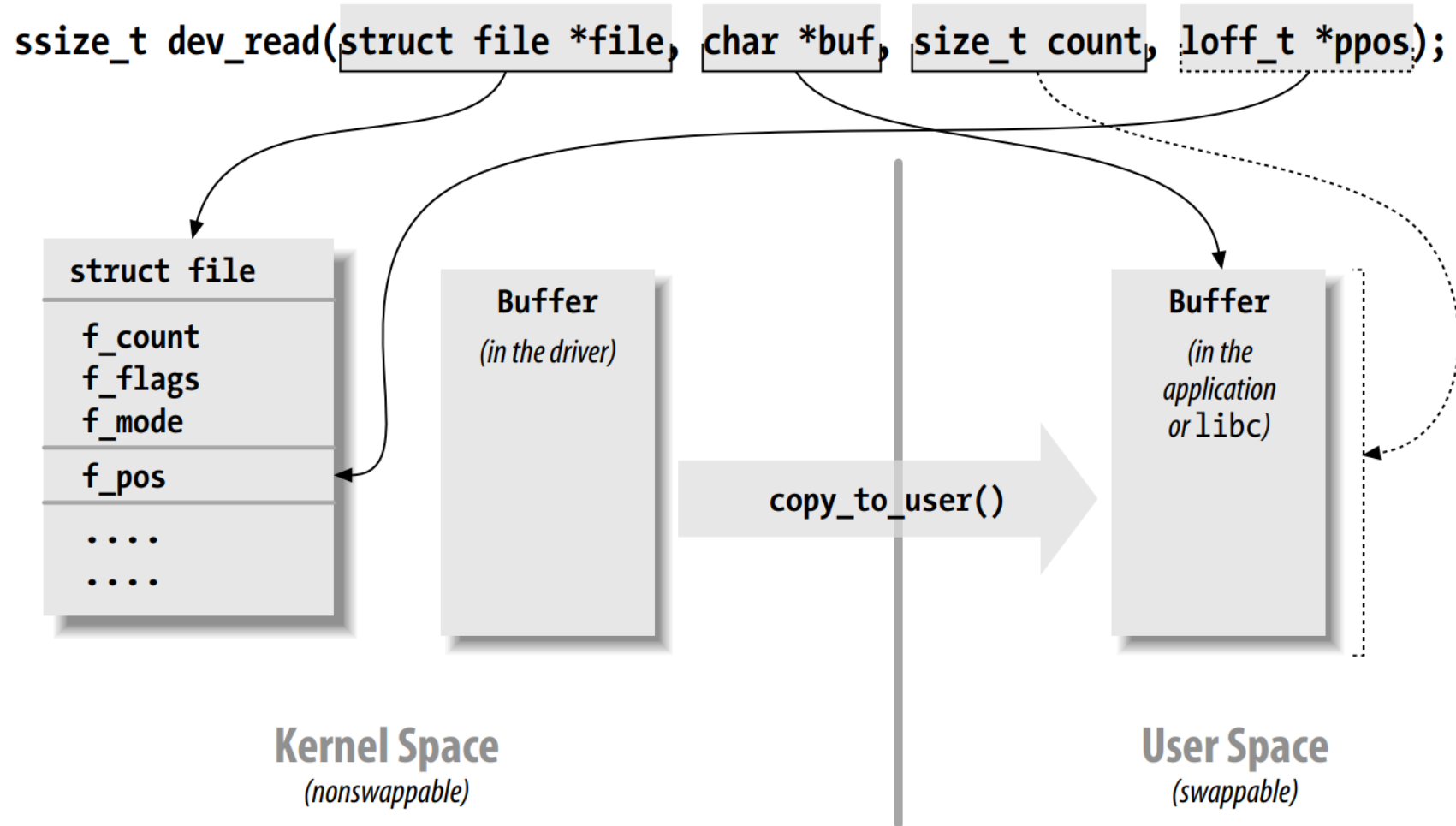


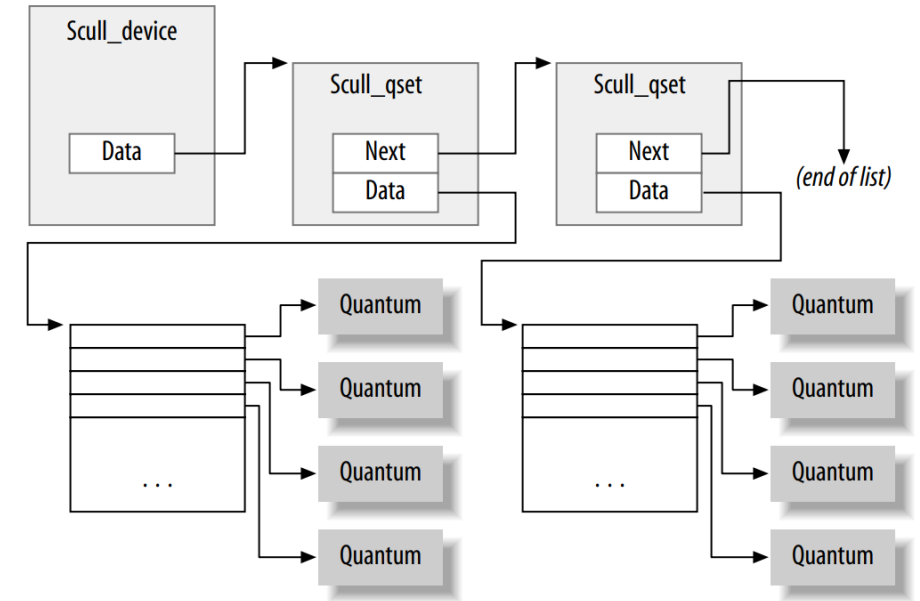
- In summary: `scull` maintains an array of struct containing data private to each device (minor number).



Note: file structs are per-process, and in kernel space. Each process can open multiple files (recall the fd array).

A Look Back at the Read() Call





Read in the scull Driver



```
/* follow the list up to the right position (defined elsewhere) */
dptr = scull_follow(dev, item);
```

```
if (dptr == NULL || !dptr->data || ! dptr->data[s_pos])
    goto out; /* don't fill holes */
```

```
/* read only up to the end of this quantum */
if (count > quantum - q_pos)
    count = quantum - q_pos;
```

May only perform a partial read, we expect more read calls will come if incomplete.

```
if (copy_to_user(buf, dptr->data[s_pos] + q_pos, count)) {
    retval = -EFAULT;
    goto out;
}
*f_pos += count;
retval = count;
```

Copy to user, return bytes read and update file position.

Example demo: what happens if we don't update file position correctly?

```
out:
    up(&dev->sem);
    return retval;
}
```

Write in the scull Driver



```
ssize_t scull_write(struct file *filp, const char __user *buf, size_t count,
                   loff_t *f_pos)
{
    struct scull_dev *dev = filp->private_data;
    struct scull_qset *dptr;
    int quantum = dev->quantum, qset = dev->qset;
    int itemsize = quantum * qset;
    int item, s_pos, q_pos, rest;
    ssize_t retval = -ENOMEM; /* value used in "goto out" statements */

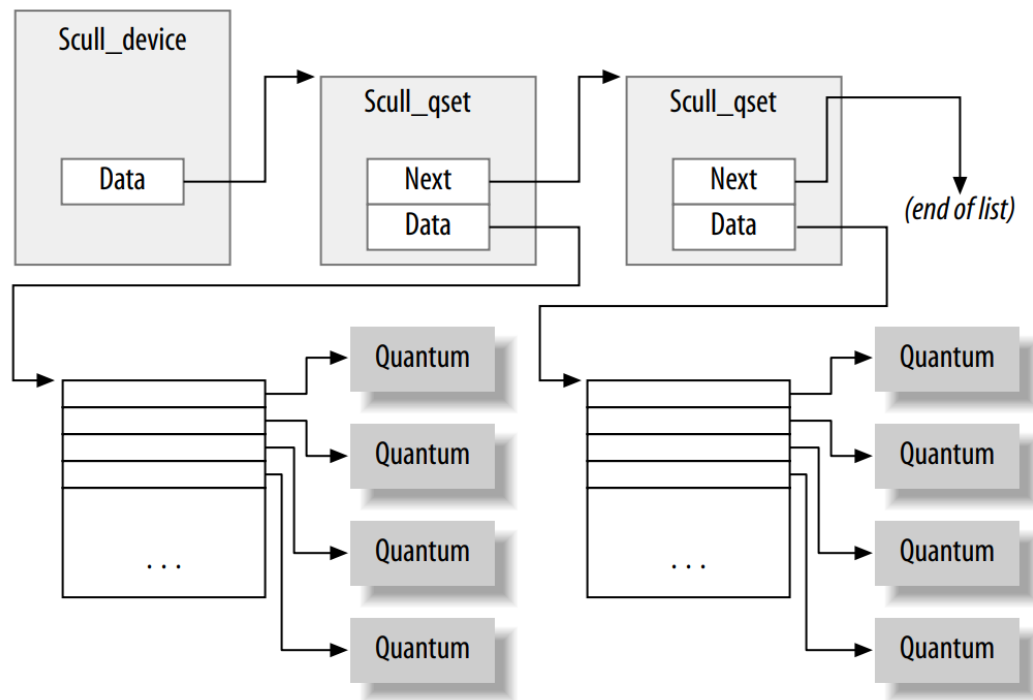
    if (down_interruptible(&dev->sem))
        return -ERESTARTSYS;

    /* find listitem, qset index and offset in the quantum */
    item = (long)*f_pos / itemsize;
    rest = (long)*f_pos % itemsize;
    s_pos = rest / quantum; q_pos = rest % quantum;

    /* follow the list up to the right position */
    dptr = scull_follow(dev, item);
    if (dptr == NULL)
        goto out;
    if (!dptr->data) {
        dptr->data = kmalloc(qset * sizeof(char *), GFP_KERNEL);
        if (!dptr->data)
```

As with read(), we first look obtain the scull_dev struct associated with current device file, perform boundary checks, acquire the lock (semaphore), and seek to right position for writing in the quantum set.

- Use `kmalloc` to allocate new array for a quantum if empty.
 - Remember to clean up any dynamically allocated kernel memory structures on exit.



```
        goto out;
        memset(dp->data, 0, qset * sizeof(char *));
    }
    if (!dp->data[s_pos]) {
        dp->data[s_pos] = kmalloc(quantum, GFP_KERNEL);
        if (!dp->data[s_pos])
            goto out;
    }
    /* write only up to the end of this quantum */
    if (count > quantum - q_pos)
        count = quantum - q_pos;

    if (copy_from_user(dp->data[s_pos]+q_pos, buf, count)) {
        retval = -EFAULT;
        goto out;
    }
    *f_pos += count;
    retval = count;

    /* update the size */
    if (dev->size < *f_pos)
        dev->size = *f_pos;

out:
    up(&dev->sem);
    return retval;
}
```

ioctl in the scull Driver



```
switch(cmd) {

#ifdef SCULL_DEBUG
    case SCULL_IOCHARDRESET:
        /*
         * reset the counter to 1, to allow unloading in case
         * of problems. Use 1, not 0, because the invoking
         * process has the device open.
         */
        while (MOD_IN_USE)
            MOD_DEC_USE_COUNT;
        MOD_INC_USE_COUNT;
        /* don't break: fall through and reset things */
#endif /* SCULL_DEBUG */

    case SCULL_IOCRESET:
        scull_quantum = SCULL_QUANTUM;
        scull_qset = SCULL_QSET;
        break;

    case SCULL_IOCSEQQUANTUM: /* Set: arg points to the value */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        ret = __get_user(scull_quantum, (int *)arg);
        break;

    case SCULL_IOTQQUANTUM: /* Tell: arg is the value */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        scull_quantum = arg;
        break;
```

```
    case SCULL_IOCQQUANTUM: /* Get: arg is pointer to result */
        ret = __put_user(scull_quantum, (int *)arg);
        break;

    case SCULL_IOCQQUANTUM: /* Query: return it (it's positive) */
        return scull_quantum;

    case SCULL_IOCXXQUANTUM: /* eXchange: use arg as pointer */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        tmp = scull_quantum;
        ret = __get_user(scull_quantum, (int *)arg);
        if (ret == 0)
            ret = __put_user(tmp, (int *)arg);
        break;

    case SCULL_IOCHQUANTUM: /* sHift: like Tell + Query */
        if (!capable(CAP_SYS_ADMIN))
            return -EPERM;
        tmp = scull_quantum;
        scull_quantum = arg;
        return tmp;

    default: /* redundant, as cmd was checked against MAXNR */
        return -ENOTTY;
}
return ret;
```



The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN